

Class 18: Performance Tuning & Monitoring

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

IIT Bhilai

March 26, 2026

Table of Contents

1. Introduction to Performance Tuning
2. Understanding the Spark UI
3. Memory Management
4. Caching & Persisting
5. Common Errors: OOM & GC Tuning

Introduction to Performance Tuning

Why Performance Tuning?

- Spark applications process massive amounts of data; inefficiencies scale up quickly.
- Default configurations are designed to be safe, not necessarily optimal.
- Poorly tuned jobs lead to:
 - High cloud infrastructure costs.
 - Missed SLAs (Service Level Agreements) for data pipelines.
 - Cluster resource contention, affecting other users.
 - Out of Memory (OOM) errors and application crashes.

The Lifecycle of Tuning

1. **Measure:** Use tools like Spark UI and logs to find bottlenecks.
2. **Analyze:** Identify the root cause (e.g., skew, GC overhead, slow I/O).
3. **Optimize:** Apply code or configuration changes.
4. **Validate:** Rerun and compare metrics to ensure improvement.

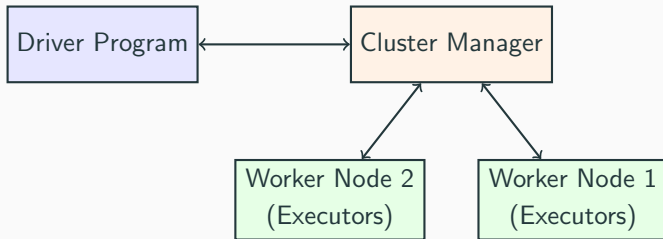
Key Performance Bottlenecks

- **CPU Bottlenecks:** Complex UDFs, excessive serialization.
- **Memory Bottlenecks:** OOM errors, excessive Garbage Collection (GC).
- **Network Bottlenecks:** Massive shuffles, large broadcast variables.
- **Disk I/O Bottlenecks:** Spilling data to disk, slow storage layers.

Hardware vs. Software Tuning

- **Hardware Tuning:**
 - Adding more nodes or more powerful instances (Scale out / Scale up).
 - Switching from HDDs to SSDs or NVMe drives.
- **Software/Application Tuning (Our Focus):**
 - Optimizing transformations and actions.
 - Managing memory efficiently.
 - Configuring Spark properly.

Spark Architecture Reminder



- We tune settings for the Driver and Executors.

Understanding the Spark UI

What is the Spark Web UI?

- Spark provides a suite of Web UIs to monitor the status and resource consumption of your cluster.
- Available by default on port **4040** of the driver node (e.g., `http://driver-node:4040`).
- If multiple contexts are running, ports 4041, 4042, etc., are used.
- **Tabs available:** Jobs, Stages, Tasks, Storage, Environment, Executors, and SQL.

- Provides a high-level summary of all jobs in the application.
- A "Job" is triggered by a Spark **Action** (e.g., `count()`, `collect()`, `write()`).
- Displays:
 - Status (Running, Succeeded, Failed).
 - Duration.
 - Progress bar of Stages (e.g., 2/4 stages completed).

Event Timeline in the Jobs Tab

- A visual timeline of events: executors added/removed, jobs started/completed.
- Helps identify **resource acquisition delays**.
- If there is a long gap before a job starts, the cluster manager (YARN, K8s) might be slow in allocating executors.

The Stages Tab

- A Job is broken into Stages. Stages are divided by **Shuffle boundaries** (wide transformations).
- The Stages tab lists all stages for a given job.
- Key metrics here:
 - **Input/Output size:** Data read/written.
 - **Shuffle Read/Write:** Data moved across the network between stages.
 - **Task count:** Number of partitions being processed.

Diagnosing Problems in the Stages Tab

- **High Shuffle Write Time:** Could indicate inefficient partitioning or massive data movement (e.g., a huge `groupByKey`).
- **Data Skew Indicator:** Look at the task summary statistics (Min, 25th, Median, 75th, Max).
- If the Max task duration is $10\times$ the Median, you have a skewed partition!

The Tasks Level View

- Clicking into a Stage reveals the Tasks view.
- Each Task processes a single partition of data.
- You can see exactly which Executor ran which Task.
- **Locality Level:** PROCESS_LOCAL, NODE_LOCAL, RACK_LOCAL, or ANY.
- We want data locality to be as close to PROCESS_LOCAL as possible.

Identifying Straggler Tasks

- A **straggler** is a task that takes significantly longer than others in the same stage.
- Causes:
 - Data skew (one partition has 90% of the data).
 - Bad hardware (the node running the task has a failing disk or high CPU load).
- Spark can use **Speculative Execution** (`spark.speculation=true`) to re-launch slow tasks on other nodes.

The Storage Tab

- Displays information about cached RDDs and DataFrames.
- Shows:
 - Storage Level (e.g., Memory Deserialized, Disk Serialized).
 - Size in Memory vs. Size on Disk.
 - Fraction of partitions cached (e.g., 100% cached).
- Use this to verify that your `cache()` calls are actually working and fitting into memory!

The Environment Tab

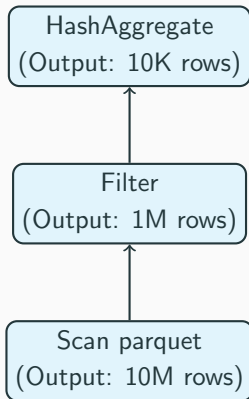
- Lists the configuration and environment variables for your Spark application.
- **Categories:**
 - JVM Information (Java version).
 - Spark Properties (`spark.executor.memory`, etc.).
 - System Properties.
 - Classpath Entries.
- Vital for debugging: "Did my configuration override actually get applied?"

The Executors Tab

- Granular view of resources across executors.
- Shows Memory Used, Disk Used, Active Tasks, Failed Tasks per Executor.
- Helps identify **hardware issues**: If Executor 3 has 500 failed tasks and others have 0, Node 3 might have a hardware problem.
- Monitor GC (Garbage Collection) time per executor here.

- Crucial for Spark SQL and DataFrame/Dataset users.
- Visualizes the physical execution plan (DAG) generated by Catalyst.
- Shows exactly how queries are executed under the hood.
- Metrics like "number of output rows" are overlayed on the DAG.

SQL Tab: Reading the DAG



- DAGs read from bottom to top.
- Shows the effect of predicate pushdown (filtering happens early).

DAG Example: Join Strategies

- The SQL tab reveals which join strategy Spark chose.
- Did it choose SortMergeJoin (expensive shuffle) or BroadcastHashJoin (fast)?
- If you expected a broadcast join but see a sort-merge join, your tables might be larger than the `spark.sql.autoBroadcastJoinThreshold`.

- What happens to the Spark UI when the application finishes? It disappears!
- **Spark History Server** reconstructs the UI of completed applications by reading event logs.
- Must enable logging:
 - `spark.eventLog.enabled = true`
 - `spark.eventLog.dir = hdfs://...`

Spark UI Summary

- The UI is your window into the cluster's soul.
- Always check the Stages tab for data skew.
- Always check the Storage tab to ensure caches fit.
- Always check the SQL tab to verify Catalyst physical plans.

Memory Management

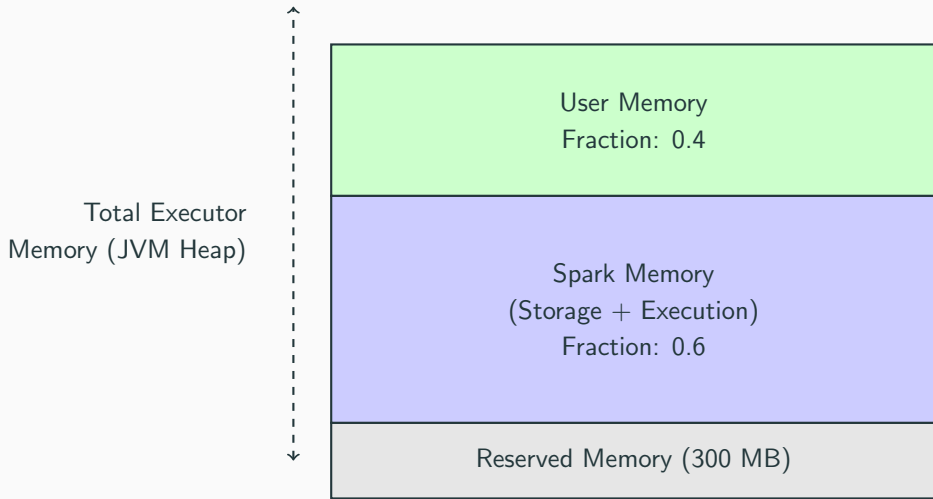
Memory Management Evolution

- **Pre-Spark 1.6: Static Memory Management**
 - Rigid boundaries between memory pools.
 - Hard to tune, frequently led to OutOfMemory (OOM) errors.
- **Post-Spark 1.6: Unified Memory Management**
 - Dynamic boundaries between Execution and Storage memory.
 - Much more robust and requires less manual tuning.
 - Now the default standard.

The Executor JVM Heap

- Executors are Java Virtual Machines (JVMs).
- Total Executor Memory is set by `spark.executor.memory` (e.g., 4G, 8G).
- The OS also has an **Overhead Memory** allowance for off-heap allocations, thread stacks, etc.
- We focus on how the JVM Heap is subdivided.

JVM Heap Divisions in Spark



1. Reserved Memory

- Hardcoded to 300 MB.
- Protected space for internal Spark objects.
- Prevents the system from crashing completely if the user asks for too much memory.
- Formula for usable heap: $\text{Usable} = \text{Total Heap} - 300\text{MB}$.

2. User Memory

- Controlled by `spark.memory.fraction` (default is 0.6 for Spark Memory, leaving 0.4 for User).
- Used for:
 - User-defined data structures (e.g., hash maps in UDFs).
 - Spark internal metadata.
 - Objects created during RDD transformations (e.g., map creating strings).
- **Warning:** Spark does not evict data from User Memory. If you create massive arrays here, you will hit an OOM.

3. Spark Memory (Execution & Storage)

- The remaining 60% of usable heap.
- This pool is shared dynamically by two components:
 1. **Execution Memory:** Used for computation (shuffles, joins, sorts, aggregations). Short-lived data.
 2. **Storage Memory:** Used for caching RDDs/DataFrames and broadcasting variables. Long-lived data.

The Dynamic Boundary

- `spark.memory.storageFraction` (default 0.5) sets the initial boundary.
- **The Magic of Unified Memory:**
 - Execution can borrow from Storage if Storage has free space.
 - Storage can borrow from Execution if Execution has free space.

Eviction Policies

- What happens if Execution needs memory back?
- **Execution Priority:** Execution memory can *evict* Storage memory blocks (spill them to disk or drop them) to reclaim space.
- What happens if Storage needs memory back?
- **Storage cannot evict Execution.** Storage must wait until Execution finishes its tasks and frees up memory naturally.
- Why? Dropping cached data just causes recomputation later. Spilling active execution data mid-task causes immediate, severe slowdowns.

When to Tune Memory Fractions

- **Decrease** `spark.memory.fraction` (< 0.6):
 - If you have heavy UDFs or create many complex custom Java/Scala objects, you need more User Memory.
- **Increase** `spark.memory.storageFraction` (> 0.5):
 - If your workload is primarily iterative machine learning with heavy caching, and very few large shuffles.

Off-Heap Memory

- Spark can allocate memory outside the JVM Heap directly to the OS.
- **Benefits:**
 - Bypasses Garbage Collection (GC) entirely.
 - Greatly reduces GC pause times for massive datasets.
- **Configuration:**
 - `spark.memory.offHeap.enabled = true`
 - `spark.memory.offHeap.size = <size>`
- Heavily utilized by Project Tungsten for binary processing.

Memory Overhead (YARN/K8s)

- Total container memory requested from the cluster manager is:
- Executor Memory + Memory Overhead.
- Overhead covers VM overheads, interned strings, native memory (like PySpark Python worker processes).
- Config: `spark.executor.memoryOverhead` (Default is 10% of executor memory, minimum 384MB).
- **Crucial Error:** "Container killed by YARN for exceeding memory limits" → Increase overhead, NOT executor memory!

Memory Management Summary

- Unified Memory dynamically shares the heap between caching and active computation.
- Execution takes priority over Storage.
- User Memory is dangerous if mismanaged by custom code.
- Off-heap memory is a powerful tool to defeat GC pauses.

Caching & Persisting

The Need for Caching

- Spark's core mechanism is **Lazy Evaluation**.
- Every action triggers a full recomputation from the source file.
- If you use a DataFrame multiple times, it will be read and transformed multiple times.
- **Caching** saves the intermediate state of an RDD/DataFrame in memory or on disk to prevent recomputation.

`cache()` vs `persist()`

- `cache()` is simply a convenience function that calls `persist()` with default settings.
- For RDDs: `cache()` defaults to `MEMORY_ONLY`.
- For DataFrames/Datasets: `cache()` defaults to `MEMORY_AND_DISK` (because Catalyst uses efficient Tungsten binary encoding).
- `persist(StorageLevel)` lets you explicitly choose how data is stored.

Understanding Storage Levels

Storage Levels are combinations of:

1. **Storage Location:** Memory, Disk, or Both.
2. **Format:** Deserialized (Java objects) or Serialized (Byte arrays).
3. **Replication:** 1 copy or 2 copies (_2).

Common Storage Levels Table

Level	Space Used	CPU Time	In Memory?
MEMORY_ONLY	High	Low	Yes
MEMORY_ONLY_SER	Low	High	Yes
MEMORY_AND_DISK	High	Medium	Some
MEMORY_AND_DISK_SER	Low	High	Some
DISK_ONLY	Low	High	No

- Stores RDD as deserialized Java objects in the JVM.
- **Pros:** Maximum speed. No overhead for serialization or deserialization.
- **Cons:** Extremely memory-intensive. High Garbage Collection (GC) pressure.
- If partitions don't fit in memory, they are dropped and recomputed on demand.

- Default for DataFrames.
- Stores in memory, but if a partition doesn't fit, it spills it to disk.
- **Pros:** Prevents expensive recomputation of dropped partitions.
- **Cons:** Disk I/O is slow.
- *Note: Sometimes recomputing a fast transformation is quicker than reading it from disk!*

Serialized Storage (_SER)

- e.g., `MEMORY_ONLY_SER`.
- Stores data as a massive byte array instead of millions of tiny Java objects.
- **Pros:** Drastically reduces memory footprint. Massive reduction in GC pauses.
- **Cons:** Costs CPU cycles to serialize and deserialize the bytes.
- **Best Practice:** Use this if `MEMORY_ONLY` causes GC issues or OOMs.

Replication (_2)

- e.g., MEMORY_AND_DISK_2.
- Keeps two copies of each partition on different nodes.
- **Pros:** High fault tolerance. If Node A crashes, task immediately resumes on Node B without recomputing data.
- **Cons:** Doubles memory usage and requires network transfer. Use only for ultra-critical fast-response pipelines.

Code Example: Caching in Scala

```
1 import org.apache.spark.storage.StorageLevel
2
3 val df = spark.read.parquet("hdfs://data/sales")
4 val expensiveDF = df.groupBy("region").sum("amount")
5
6 // Cache using default
7 expensiveDF.cache()
8
9 // Persist with specific level
10 val filteredDF = df.filter($"amount" > 1000)
11 filteredDF.persist(StorageLevel.MEMORY_AND_DISK_SER)
12
13 // Materialize the cache (Lazy evaluation!)
14 expensiveDF.count()
15
16 // Always clean up when done!
17 expensiveDF.unpersist()
```

Code Example: Caching in Python (PySpark)

```
1 from pyspark import StorageLevel
2
3 df = spark.read.parquet("hdfs://data/sales")
4 expensive_df = df.groupBy("region").sum("amount")
5
6 # Cache using default
7 expensive_df.cache()
8
9 # Persist with specific level
10 filtered_df = df.filter(df.amount > 1000)
11 filtered_df.persist(StorageLevel.MEMORY_AND_DISK_2)
12
13 # Materialize the cache
14 expensive_df.count()
15
16 # Always clean up when done!
17 expensive_df.unpersist()
```


When NOT to Cache

- **Caching is not free!** It consumes memory and CPU.
- Do NOT cache if:
 - You only use the DataFrame once.
 - The dataset is tiny (recomputing takes milliseconds).
 - The transformations are very cheap (e.g., simple filters), making recomputation faster than disk reads.

- Spark uses a Least Recently Used (LRU) cache policy to evict old partitions.
- However, waiting for LRU eviction is inefficient.
- **Best Practice:** Explicitly call `df.unpersist()` immediately after you are finished using a cached dataset.
- This frees up Spark Memory for Execution pools.

Caching Summary

- Use `cache()` for iterative workflows (Machine Learning).
- Prefer `MEMORY_AND_DISK` for safety.
- Prefer Serialized formats to combat GC pressure.
- Always `unpersist()`.

Common Errors: OOM & GC Tuning

Out of Memory (OOM) Errors

- OOM is the nemesis of big data engineers.
- It means the JVM tried to allocate an object but the heap was full.
- OOM can happen in two places, requiring different fixes:
 1. **Driver OOM**
 2. **Executor OOM**

Driver OOM: Causes and Fixes

- **Cause 1:** `collect()`
 - `df.collect()` pulls all data from the cluster to the single Driver node. If the data is 10GB and Driver has 2GB RAM → OOM.
 - **Fix:** Use `take(n)`, `show()`, or write directly to storage.
- **Cause 2: Massive Broadcasts**
 - Broadcasting a very large table (e.g., > 2GB) exhausts driver memory.
 - **Fix:** Disable auto-broadcast, use `SortMergeJoin` instead.

Executor OOM: Causes and Fixes

- **Cause 1: Data Skew**

- One task gets a massive partition that exceeds executor memory.
- **Fix:** Salting (adding random keys to distribute data).

- **Cause 2: High Concurrency**

- E.g., 5 cores per executor means 5 tasks share the heap.
- **Fix:** Reduce `spark.executor.cores` to 2 or 3, increasing memory per task.

- **Cause 3: Inefficient UDFs**

- Creating large maps/arrays for every row inside a map function.
- **Fix:** Optimize UDF logic or use Spark SQL native functions.

Code Example: Handling Skew (Salting) Python

```
1 from pyspark.sql.functions import rand, col, lit, concat
2
3 # df1 has heavily skewed key "join_key" (e.g., nulls or "unknown")
4 # 1. Add a random salt (0 to 9) to the skewed key
5 salted_df1 = df1.withColumn("salt", (rand() * 10).cast("int")) \
6                 .withColumn("salted_key", concat(col("join_key"), lit("_"), col("salt")))
7
8 # 2. For df2 (the smaller table), we must explode it to match all 10 salts
9 # (Assuming logic to cross join df2 with array(0..9))
10 exploded_df2 = ...
11
12 # 3. Join on the new uniformly distributed key
13 joined_df = salted_df1.join(exploded_df2, "salted_key")
```


Garbage Collection (GC) Fundamentals

- JVM manages memory automatically via Garbage Collection.
- **Minor GC:** Fast cleanup of short-lived objects (Young Generation). Minimal impact.
- **Major GC (Full GC):** Cleans up long-lived objects (Old Generation).
- **Stop-the-World:** During Full GC, all application execution halts!
- If Full GC takes 30 seconds, your tasks pause, and network connections may timeout.

Detecting GC Issues

- Check the Spark UI Stages tab.
- Look at "Task Time" vs "GC Time".
- **Rule of Thumb:** If GC Time $> 10\%$ of Task Time, you have a GC problem.
- Symptoms: Erratic task durations, frequent `ExecutorLostFailure` due to heartbeats timing out during GC pauses.

- **1. Reduce Object Creation:**

- Use Spark SQL/DataFrames instead of RDDs. Catalyst uses Tungsten, managing memory explicitly as bytes, bypassing JVM GC entirely.
- Use primitive types (`Int`, `Double`) instead of objects (`String`, custom classes) if using RDDs.

- **2. Use Serialized Caching:**

- `MEMORY_ONLY_SER` stores one large byte array instead of millions of tiny objects.

GC Tuning: Switching Algorithms

- By default, Java 8 uses Parallel GC.
- For large heaps (> 8 GB), **G1GC (Garbage First GC)** provides much more predictable and shorter pause times.
- **Configuration:**
 - Add to spark-submit:
 - `--conf "spark.executor.extraJavaOptions=-XX:+UseG1GC"`
- *Note: Java 11+ uses G1GC by default.*

- If you are still using RDDs, the default Java Serializer is slow and bloats data size.
- Switch to **Kryo Serializer**. It is 10x faster and more compact.
- **Configuration:**
 - `spark.serializer = org.apache.spark.serializer.KryoSerializer`
- Requires registering custom classes for maximum performance.

Summary: The Tuning Checklist

1. Understand the data shape (Skew?).
2. Use DataFrames and Spark SQL, let Catalyst optimize.
3. Check Spark UI for slow stages and high GC time.
4. Manage memory overhead to avoid YARN kills.
5. Cache intelligently, unpersist eagerly.
6. Optimize Shuffles (Broadcast vs SortMerge).

Thank You!

Any Questions?